



Bibliotheks-Terminal

Vollständige Projektdokumentation

C# · Entity Framework · Docker · SQL Server

1. Projektüberblick

Das Bibliotheks-Terminal ist eine C#-Konsolenanwendung zur Verwaltung von Büchern, Autoren, Verlagen und Orten. Die Daten werden in einer SQL-Server-Datenbank gespeichert, die automatisch per Docker gestartet wird — der Benutzer muss nur eine einzige .exe starten, alles andere passiert automatisch.

Zweck	Verwaltung einer Bibliothek: Bücher, Autoren, Verlage und Orte anlegen, bearbeiten und löschen
Technologie	C# (.NET 8) · Entity Framework Core · SQL Server 2022 · Docker
Besonderheit	Eine einzige .exe startet automatisch den Docker-Container und wartet bis die Datenbank bereit ist

2. Projektstruktur — Welche Datei macht was?

Das Projekt besteht aus einer einzigen Solution mit einem Projekt. Alle Dateien liegen im selben Ordner:

Datei	Zweck
Program.cs	Einstiegspunkt — startet Docker, initialisiert DB, Hauptmenü
DockerLauncher.cs	Startet Docker Compose und wartet bis SQL Server bereit ist
BibliothekContext.cs	Entity Framework Kontext — Verbindung zur Datenbank
Bibliothek.cs	Gesamte App-Logik: Anzeige, Erstellen, Löschen, Bearbeiten
Buch.cs	Datenmodell: Buch (Titel, ISBN, Seiten, Autor, Verlag)
Autor.cs	Datenmodell: Autor (Name, Jahrgang, Bücher)
Verlag.cs	Datenmodell: Verlag (Name, Ort, Telefon, Email, Bücher)
Ort.cs	Datenmodell: Ort (Name, PLZ, Verlage)
docker-compose.yml	Docker-Konfiguration für den SQL Server Container
Bibliothek.csproj	Projektdatei — NuGet-Pakete, .NET Version, Publish-Einstellungen

3. Programmstart — Schritt für Schritt

Wenn der Benutzer die .exe startet, passiert folgendes in genau dieser Reihenfolge:

Schritt 1 — DockerLauncher startet den SQL Server

Die allererste Zeile in Program.cs lautet:

```
await DockerLauncher.StartAndWaitAsync();
```

DockerLauncher.cs führt im Hintergrund folgenden Befehl aus:

```
docker compose up -d
```

Das `-d` steht für "detached" — der Container läuft im Hintergrund, das Konsolenfenster bleibt frei. Docker liest dabei die `docker-compose.yml` und startet den SQL Server Container namens `MeinSQLServer` auf Port 1433.

Falls Docker nicht installiert ist oder nicht läuft, erscheint eine rote Fehlermeldung und das Programm beendet sich sofort mit `Environment.Exit(1)`.

Schritt 2 — Warten bis die Datenbank bereit ist

Docker startet den Container schnell, aber SQL Server selbst braucht einige Sekunden zum Hochfahren. Deshalb prüft `DockerLauncher` in einer Schleife alle 2 Sekunden ob die Verbindung klappt:

```
using var ctx = new BibliothekContext();
if (!await ctx.Database.CanConnectAsync())
    throw new Exception("Nicht erreichbar");
```

`CanConnectAsync()` gibt `true` zurück wenn SQL Server antwortet — das ist wichtig, denn ohne die Prüfung auf den Rückgabewert würde das Programm sofort weitermachen, auch wenn die DB noch nicht bereit ist. Das Programm wartet maximal 60 Sekunden. Danach bricht es mit einer Fehlermeldung ab.

Schritt 3 — Datenbank erstellen falls nicht vorhanden

Jetzt übernimmt `Program.cs` und erstellt bei Bedarf alle Tabellen:

```
context.Database.EnsureCreated();
```

`EnsureCreated()` schaut ob die Datenbank `BibliothekDB` bereits existiert. Falls nicht, erstellt Entity Framework automatisch alle Tabellen (Buecher, Autoren, Verlage, Orte) anhand der Klassen und Beziehungen die in `BibliothekContext.cs` definiert sind.

Schritt 4 — Beispieldaten laden falls die DB leer ist

Beim allerersten Start ist die Datenbank leer. Das Programm prüft das und füllt sie mit Testdaten:

- 3 Orte: Berlin, Hamburg, Klagenfurt
- 3 Verlage: Nachbar (Berlin), Gute Pucher (Hamburg), Pücherwurm (Klagenfurt)
- 3 Autoren: Roland Hraschan, Chuck Norris, Timmy
- 3 Bücher — je ein Buch pro Autor

Bei jedem weiteren Start überspringt das Programm diesen Schritt, da die Daten bereits vorhanden sind.

Schritt 5 — Das Hauptmenü erscheint

Ab jetzt läuft das Programm in einem `do-while`-Loop bis der Benutzer `q` drückt. Bei jedem Durchlauf wird das Menü neu gezeichnet und auf eine Tasteneingabe gewartet (`ReadKey` — kein Enter nötig).

4. Datenmodell — Die vier Klassen

Das Projekt hat vier Datenklassen (Models). Entity Framework übersetzt diese automatisch in SQL-Tabellen:

Klasse	Felder	Beziehung
Buch	Titel, ISBN (PK), AnzahlSeiten	→ 1 Autor, → 1 Verlag
Autor	Id, Name, Jahrgang	→ viele Bücher
Verlag	Id, Name, Telefon, Email	→ 1 Ort, → viele Bücher
Ort	Id, Name, Postleitzahl	→ viele Verlage

Beziehungen zwischen den Klassen

Die Klassen sind miteinander verknüpft. Eine falsche Reihenfolge beim Löschen würde Fremdschlüssel-Fehler auslösen — deshalb wurde in BibliothekContext.cs SetNull konfiguriert:

Wenn gelöscht wird...	Was passiert mit verknüpften Daten
Autor löschen	Bücher bleiben — Autor-Feld wird null (SetNull)
Verlag löschen	Bücher bleiben — Verlag-Feld wird null (SetNull)
Ort löschen	Verlage bleiben — Firmensitz-Feld wird null (SetNull)

SetNull bedeutet: Wenn z.B. ein Autor gelöscht wird, behalten seine Bücher alle Daten — nur das Autor-Feld in der Bücher-Tabelle wird auf NULL gesetzt. Die Bücher verschwinden nicht.

Besonderheit: ISBN als Primärschlüssel

Bei der Klasse Buch ist nicht die automatisch vergebene Id der Primärschlüssel, sondern die ISBN:

```
[Key]
public string ISBN { get; set; } = string.Empty;
```

Das bedeutet: Zwei Bücher mit derselben ISBN können nicht gleichzeitig in der Datenbank existieren. Entity Framework erzwingt das automatisch über einen UNIQUE-Index.

5. Datenbankverbindung — BibliothekContext.cs

BibliothekContext erbt von DbContext (Entity Framework). Er ist die einzige Stelle im gesamten Projekt wo der Verbindungsstring steht:

```
Server=localhost,1433;Database=BibliothekDB;
User Id=sa;Password=Passwort123!;TrustServerCertificate=True;
```

Diese Verbindung wird sowohl von DockerLauncher (Health-Check) als auch von Program.cs und allen Methoden in Bibliothek.cs verwendet — überall wird new BibliothekContext() aufgerufen, und der Verbindungsstring kommt automatisch aus OnConfiguring().

6. Das Hauptmenü — Welche Taste macht was?

Taste	Aktion	Aufgerufene Methode
1	Bücherübersicht	Bibliothek.BuchÜbersicht(context)
2	Autorenübersicht	Bibliothek.AutorÜbersicht(context)
3	Verlagsübersicht	Bibliothek.VerlagÜbersicht(context)
4	Ortsübersicht	Bibliothek.OrtÜbersicht(context)
+	Neues Element	Bibliothek.Erstellen(context)
-	Element löschen	Bibliothek.Löschen(context)
*	Element bearbeiten	Bibliothek.bearbeiten(context)
r	DB zurücksetzen	Direkt in Program.cs — alle Tabellen leeren
q	Beenden	Loop verlässt, SaveChanges(), Programm endet

7. Bibliothek.cs — Die gesamte App-Logik

Bibliothek.cs enthält alle statischen Methoden für die eigentliche Funktionalität. Keine Methode speichert selbst in der Datenbank — das übernimmt immer Program.cs mit `context.SaveChanges()` am Ende jedes Menü-Durchlaufs.

Übersichten (BuchÜbersicht, AutorÜbersicht, ...)

Laden alle Datensätze inkl. verknüpfter Objekte via `Include()` und geben sie formatiert in der Konsole aus. `Include()` ist wichtig — ohne es würden z.B. bei Büchern die Autoren-Daten nicht mitgeladen (Lazy Loading ist in Entity Framework standardmäßig aus).

Erstellen

Fragt den Benutzer nach allen nötigen Feldern. Bei Fremdschlüsseln (z.B. welcher Verlag für ein Buch) werden alle vorhandenen Einträge als nummerierte Liste angezeigt und der Benutzer wählt per Nummer.

Löschen — Besonderheit Chuck Norris

Beim Löschen eines Autors gibt es ein Easter Egg: Chuck Norris kann nicht gelöscht werden. Der Code prüft den Namen, gibt eine Warnung aus und beendet das Programm nach einer dramatischen Sequenz mit `Environment.Exit(0)`. Das ist absichtlich so eingebaut.

Bei allen anderen Löschvorgängen fragt das Programm was mit verknüpften Daten passieren soll — mitlöschen oder behalten (ohne Referenz).

Bearbeiten

Zeigt zuerst eine Liste aller vorhandenen Einträge, lässt den Benutzer einen auswählen, und zeigt dann ein Untermenü welches Feld geändert werden soll. Der aktuelle Wert wird dabei immer angezeigt. Nur das gewählte Feld wird überschrieben, alles andere bleibt gleich.

8. Docker-Konfiguration — docker-compose.yml

Die docker-compose.yml liegt neben der .exe und definiert den SQL Server Container:

```
services:
  sql-server:
    image: mcr.microsoft.com/mssql/server:2022-latest
    container_name: MeinSQLServer
    environment:
      - ACCEPT_EULA=Y
      - MSSQL_SA_PASSWORD=Passwort123!
    ports:
      - "1433:1433"
```

Port 1433 ist der Standard-SQL-Server-Port. Die linke 1433 ist der Port am Windows-PC, die rechte 1433 der Port im Container — beide müssen übereinstimmen damit der Verbindungsstring funktioniert.

9. Publishen — Eine einzige .exe erstellen

Mit folgendem Befehl wird das Projekt als selbst-enhaltende Windows-EXE gebaut (kein .NET auf dem Zielrechner nötig):

```
dotnet publish -c Release -r win-x64 --self-contained -o ./publish
```

Danach muss die docker-compose.yml in denselben Ordner wie die .exe kopiert werden:

```
copy docker-compose.yml ./publish/
```

Der publish-Ordner enthält dann alles was der Benutzer braucht — er muss nur docker-compose.yml und die .exe in einen Ordner legen und die .exe starten.

Voraussetzungen auf dem Zielrechner

- Docker Desktop installiert und gestartet
- Windows 64-Bit
- Kein .NET oder SQL Server nötig — beides ist automatisch vorhanden

10. Gesamter Programmablauf — Zusammenfassung

Schritt	Was passiert	Wo im Code
1	Benutzer startet .exe	Betriebssystem
2	DockerLauncher führt docker compose up -d aus	DockerLauncher.cs → StartAndWaitAsync()
3	Warten bis SQL Server auf Port 1433 antwortet	DockerLauncher.cs → while-Schleife

		mit CanConnectAsync()
4	EnsureCreated() erstellt Tabellen falls nicht da	Program.cs → context.Database.EnsureCreated()
5	Beispieldaten einfügen falls DB leer	Program.cs → if (!context.Orte.Any() ...)
6	Hauptmenü-Loop startet	Program.cs → do { ... } while (auswahl != 'q')
7	Tastendruck → passende Methode in Bibliothek.cs	Program.cs switch → Bibliothek.XyzMethode(context)
8	SaveChanges() nach jedem Durchlauf	Program.cs → context.SaveChanges()
9	q gedrückt → letztes SaveChanges() → Programm endet	Program.cs → Ende